BÀI TẬP 3

CORE ENGINE VÀ SNORT RULE MATCHING CHO HỆ THỐNG IDS

1. Mục tiêu

Xây dựng Core Engine cho IDS dựa trên signature/rule. Engine nhận dữ liệu chuẩn hóa từ Preprocessor và thông tin trạng thái/thống kê từ Flow/Connection Tracker, sau đó so khớp với bộ rule trong phạm vi được hỗ trợ để sinh Detection Event. Core Engine không đọc trực tiếp raw packet từ Scapy/PyShark.

2. Yêu cầu chung

- Tái sử dụng data model của Bài tập 1-2. Input tối thiểu gồm normalized event và flow context.
- Sử dụng bộ rule của Snort; rule/keyword ngoài phạm vi phải được báo unsupported, không làm chương trình dừng.
- Tách phần nạp/parse/index rule khỏi phần đánh giá event; không hard-code signature trong source code.
- Kết quả match phải được biểu diễn thành Detection Event thống nhất để Alert/Logging/Response có thể sử dụng ở bài tiếp theo.

3. Phạm vi Snort rule bắt buộc

Nhóm	Thành phần/keyword tối thiểu
Header	alert; tcp/udp; src/dst IP: any, IPv4, CIDR; port: any, single, range; direction: ->, <>
Identification	msg, sid, rev
Content	content, nocase, offset, depth, distance, within
Packet/TCP	flags, dsize
Flow	flow: to_server, to_client, established, not_established
Classification	classtype, priority

Không bắt buộc: pcre, byte_test/byte_jump, file inspection, TLS decryption, Lua/plugin, preprocessors hoặc toàn bộ cú pháp Snort

4. Các module của Core Engine

4.1. Rule Loader

- Đọc một hoặc nhiều file .rules; bỏ dòng trống/comment. Chuyển rule hợp lệ về Rule Parser & Validator.
- Cho phép enable/disable rule hoặc file rule bằng cấu hình. Lỗi một rule không làm dừng việc nạp các rule khác (đưa ra cảnh báo trong quá trình nạp).

4.2. Rule Parser & Validator

- Tách Rule Header và Rule Options. Chuyển rule text thành Internal Rule Object thống nhất.
- Kiểm tra cú pháp, protocol/keyword được hỗ trợ, kiểu dữ liệu và các trường bắt buộc. Phân loại valid / invalid / unsupported với lý do.

4.3. Rule Repository / Indexer

- Lưu các rule đã validate và tạo chỉ mục để chọn candidate rules theo protocol/port hoặc tiêu chí tương đương.
- Không bắt buộc thuật toán tối ưu phức tạp, nhưng không nên duyệt toàn bộ rule nếu có thể loại nhanh bằng header.

4.4. Rule Matching Engine

Mục tiêu: trả lời một rule có thỏa mãn normalized event + flow context hay không. Có thể tổ chức các matcher nội bộ, nhưng được xem là một module duy nhất trong bài tập.

Nhóm kiểm tra	Yêu cầu
Header	So khớp protocol, src/dst IP, src/dst port và direction; hỗ trợ any, CIDR, port range theo phạm vi trên.
Flow/State	Dùng Flow Tracker để kiểm tra to_server/to_client, established/not_established.
Content	Tìm content trong payload/application data; hỗ trợ nocase và các giới hạn vị trí offset/depth/distance/within.
Packet/TCP	So khớp TCP flags và dsize.
Application data	Được phép dùng dữ liệu HTTP/DNS/SMTP đã parse/decode/normalize; không yêu cầu giải mã TLS.
Kết quả	Trả MATCH/NO_MATCH và thông tin match cần thiết, ví dụ sid, matched content/field.

4.5. Rule Evaluation Engine

- Nhận event + flow context, truy vấn Repository/Indexer để lấy candidate rules và gọi Rule Matching Engine cho từng candidate.
- Cho phép nhiều rule cùng match một event; không dừng ở match đầu tiên trừ khi cấu hình của bài làm nêu rõ và chứng minh hợp lý.
- Theo dõi tối thiểu: số event đã đánh giá, candidate rules, số lần match; có thể dùng để kiểm tra hiệu năng.

4.6. Detection Event Generator

- Sinh Detection Event khi rule match. Không in cảnh báo trực tiếp trong Matching Engine.
- Event tối thiểu gồm: timestamp, sid, rev, msg, priority/classtype (nếu có), src/dst IP/port, protocol, application_protocol, flow_id và thông tin match.

5. Pipeline và interface đầu ra

Pipeline bắt buộc:

Normalized Event + Flow Context → Evaluation Engine → Repository/Indexer → Matching Engine → Detection Event

Field tối thiểu	Ví dụ
sid / rev	100001 / 1
msg / priority	Directory traversal detected / 1
5-tuple + protocol	10.0.0.5:52133 → 192.168.1.20:80 / TCP
application_protocol / flow_id	HTTP / F123
matched information	content="../" hoặc field tương ứng

7. Test cases bắt buộc

#	Test case	Kết quả mong đợi
1	Nạp file có rule hợp lệ, comment và dòng trống	Chỉ rule hợp lệ được đưa sang parser.
2	Rule sai cú pháp	Đánh dấu invalid, có lý do; engine tiếp tục chạy.

#	Test case	Kết quả mong đợi
3	Rule dùng keyword ngoài phạm vi	Đánh dấu unsupported; không crash.
4	Parse header TCP + any/single IP/CIDR + port/range	Internal Rule Object đúng.
5	Direction -> và <>	So khớp đúng chiều/bidirectional.
6	Indexer với IP, TCP, UDP và any	Trả đúng tập candidate rules.
7	Header không match	Rule bị loại trước content matching.
8	flow:to_server,established	Match đúng theo Flow Tracker.
9	content có/không có nocase	Phân biệt/không phân biệt hoa thường đúng.
10	offset/depth	Chỉ match trong vùng được chỉ định.
11	Hai content với distance/within	Relative matching đúng.
12	flags và dsize	Match/không match đúng theo event.
13	Một event match đồng thời ≥2 rule	Sinh đầy đủ các kết quả match.
14	Detection Event	Có đủ sid/msg/flow/5-tuple/protocol/match info.

10. Yêu cầu nộp bài

1. **Sử dụng GitHub**
   - Mỗi sinh viên phải sử dụng **một repository GitHub riêng** cho toàn bộ bài tập. Toàn bộ bài tập lớn chỉ sử dụng 1 đường link github duy nhất này.
   - Repository phải được sử dụng ngay từ khi bắt đầu làm bài.
   - Project phải để ở chế độ công khai, source code đặt ở nhánh `main`. Nhánh `main` là nhánh mặc định.
   - Đặt tên cho repo: **Mã lớp_Họ & tên_MSSV**.
2. **Yêu cầu commit**
   - Sau khi hoàn thành mỗi task, sinh viên phải commit mã nguồn ngay (Task là một công việc nhỏ, độc lập, ví dụ: xây dựng module thu thập packet, trích xuất feature, phát triển dns/http/tcp/udp parser, phát hiện port scan, tạo chức năng cảnh báo...).
   - Sau khi thực hiện mỗi test case, sinh viên phải commit kết quả kiểm thử và các thay đổi liên quan. Tạo một thư mục **TEST/BT3** trong project để upload kết quả các testcase.
   - Mỗi commit chỉ được chứa nội dung liên quan đến một task hoặc một test case.
   - **Không được commit dồn nhiều task hoặc nhiều test case trong một commit.**
   - Không được chờ hoàn thành nhiều task rồi mới đưa mã nguồn lên GitHub.
3. **Nội dung commit**
   - Commit message phải ngắn gọn và mô tả đúng công việc đã thực hiện.
   - Ví dụ:
     - `Implement packet capture`
     - `Add port scan detection`
     - `Test port scan - case 01`
     - `Fix DNS detection false positive`
4. **Bảo toàn lịch sử**
   - Không được xóa, sửa hoặc làm lại lịch sử commit để che giấu quá trình phát triển.
   - Giảng viên có quyền kiểm tra toàn bộ **Git history** trong quá trình chấm bài.
5. **Sử dụng công cụ AI**
   - Nếu sử dụng AI, sinh viên phải ghi rõ công cụ, mục đích sử dụng và phần mã nguồn có sử dụng AI trong `README.md`.
   - Sinh viên phải hiểu và có khả năng giải thích mã nguồn mình đã nộp.

6. Lưu ý trong quá trình làm bài

- Git history, mức độ tương đồng giữa mã nguồn của các sinh viên và khả năng giải thích mã nguồn sẽ được sử dụng để đánh giá tính trung thực của bài làm.
- Giảng viên sẽ thực hiện vấn đáp trong các buổi học.
- Mọi hành vi gian lận hoặc không trung thực sẽ bị 0 điểm quá trình.